

一、个人信息

姓名	郑建辉	邮箱	xechoz.zheng@gmail.com
技术方向	移动端架构、大前端基础设施、跨模块通信、状态管理	手机	18825111003
教育背景	华南师范大学 / 本科 / 计算机科学与技术 / 2011.09 - 2015.06		

二、个人简介

10 年移动端架构经验，专注**海外**大型社交/游戏 App 的模块化治理与跨端基础设施。就职于 **欢聚时代 (YY 直播)**，主导 HAGO（海外千万级月活）从单体到 100+ 模块的架构演进，覆盖主页、直播间等核心场景。擅长模块化通信、MVVM 业务架构、状态管理、双端一致性网络层等平台能力沉淀。

三、专业技能

- **业务架构**：MVVM、模块化设计、跨模块通信、直播间容器化架构、单向数据流
- **Android UI**：Compose + 传统 View 统一状态管理
- **跨端开发**：2 年 iOS 开发经验，双端网络库设计、iOS 路由组件、iOS 图片资源优化
- **编程语言**：Kotlin / Java / 协程

四、工作经历

4.1 欢聚时代 (JOYY) / 高级 Android 工程师 (T3-1) / 2018 - 2026.6

HAGO——海外休闲游戏社交 App，覆盖语聊房、直播、休闲小游戏等场景，**月活千万级**。

- 2018-2020 业务基础架构：MVVM 架构、App 主页与主模块架构
- 2020-2022 模块化与跨模块通信：服务发现组件、网络组件、直播间架构
- 2023-2024 双端一致性网络库、iOS 路由组件、iOS 资源优化、多语言翻译流程自动化
- 2025 状态管理框架

4.2 荔枝 FM / Android 开发工程师 / 2017 - 2018

音频/直播内容平台，**月活千万级**。负责直播间 MVP 架构设计与 MVP IDE 代码生成插件。

4.3 唯品会 / Android 开发工程师 / 2015 - 2017

电商 App Android 模块开发。

五、项目经验

5.1 单向数据流状态管理框架 (2025)

背景 直播间、主播主页、多级导航等复杂场景下，传统 KVO / EventBus / LiveData 刷新粒度过大，易引起 UI 冗余刷新；协程生命周期依赖手动管理，频繁造成内存泄漏。过往项目中，此类问题反复成为崩溃和卡顿的根因，缺乏全局一致的解决方案。

方案 设计细粒度状态订阅机制，状态按属性分发而非整对象通知，从源头上消除冗余刷新。协程生命周期与状态管理器自动绑定，开发者无需手动处理，杜绝内存泄漏。框架与已有的

MVVM、Service 体系原生集成，跨页面/跨模块共享同一套 setState 更新规范，不引入额外的学习成本。

成果 Compose 与传统 View 统一状态消费 API，一套 ViewModel 在两种 UI 范式下行为完全一致，迁移零摩擦。已接入直播间、主页、导航等核心模块，内存泄漏相关崩溃率显著下降，UI 刷新频率降低约 40%。

5.2 Android & iOS 双端网络层（2023-2024）

背景 HAGO 面向东南亚、中东、南美等多市场，Android 与 iOS 各自维护独立网络层，接口定义靠文档传递，协议变更后双端对齐联调耗时数天。业务迭代加快后，网络层成为跨端协作的最大瓶颈。

方案 以 Protobuf 协议文件作为唯一真相源，编写代码生成工具自动产出 Android 与 iOS 两端网络调用代码。统一状态码体系与错误处理规范，双端在异常失效、重试、降级等场景下行为一致，业务方无需感知平台差异。生成代码与 proto 定义严格一一对应，调用方直接使用类型安全的 Request/Response。

成果 覆盖 Android 与 iOS 全部核心业务模块。协议变更只需更新 proto 文件并重新生成，联调效率提升 70% 以上，彻底消除了两端接口不一致导致的生产事故。

5.3 直播间容器化架构（2020-2022）

背景 HAGO 直播间需承载语聊房、直播、小游戏等多种业务形态。早期新增一种形态需全量回归，不同形态代码深度耦合；业务要求各种房间形态间快速切换，但传统方案需退出-加载-重连，体验割裂。

方案 设计容器化架构，每种业务形态作为独立插件模块，编译期按需组合，未使用插件零体积引入。定义统一的宿主-插件通信协议，插件只需实现标准 Service 接口，宿主自动管理加载、生命周期与资源回收。基于 5.4 的 Service Manager，插件注册与发现完全自动化。

成果 20+ 房间模块独立开发测试、互不干扰。支持不退出直播间秒级热切换业务形态，消除「退出-加载-重连」体验断层。新增房间形态无需修改宿主代码，迭代周期从周级缩短到天级。

5.4 跨模块通信与 MVVM 架构框架（2018-2022，贯穿至今）

HAGO 架构基座，后续所有项目（直播间、网络层、状态管理）均构建于此框架之上。

背景 HAGO 业务从单体扩张到 100+ 模块后，面临两大核心问题：模块间如何规范通信，模块内如何用一致的结构组织子功能与共享数据。彼时 MVVM 普遍绑定页面生命周期，同一逻辑在不同页面需各写一套，模块内部结构随开发者习惯各异，代码复用和交接成本极高。

方案 模块间——Service Manager：基于 KSP 编译期生成服务注册与发现代码，避免运行时反射开销，接口与实现编译期即可校验。模块内——MVVM 统一架构：ViewModel 绑定业务 Context 而非页面生命周期，为各模块提供一致的子功能组织与数据共享范式，同一逻辑在 Activity、Fragment、直播间容器中自由复用。

成果 团队 20+ 开发者日常使用，新增模块只需定义接口与实现，框架自动完成注册分发。模块间由人肉约定转为编译期保障，模块内结构统一、交接成本大幅降低。该框架持续演进至 2025 年，承载了 HAGO 全部 100+ 业务模块的通信与架构规范。